

[Das...](#) / [My ...](#) / [C...](#) / [Object Orientation: Paradigm and Pro...](#) / [Last Year's Quiz on Data Abstracti...](#)

Question **1**

Not yet answered

Marked out of 30.00

Answer the following questions in precise, unambiguous and objective manner **consistent with your implementation**. If you need to give code in the answers, give it inline or as a box separately in the same pdf as the answers.

1. Give the list of the names of your header files that have imperative code and the names of your header files without imperative code.
2. Give the names of pairs of modules that are independent of each other--here independence means, as long as the (global and method) function prototypes are not changed, the internals of function bodies in each module in a pair of independent modules can be changed without affecting the other module.
3. Do your headers reveal a lot of the data (storage) structures and implementation of their properties? If yes, is it necessary? If yes, why, or If not, how can it be changed?
4. Can you use other classes in place of the <Type> parameter for templates or for the base element type, not just primitive types?
5. Can we make a list (and hence a queue, a deque, a heap) made up of elements of different types using your implementation? Let's call it a heterogeneous list.
6. Can you answer yes to Q5 without answering yes first to Q4?
7. For each of your classes (or structs), give a score to your implementation depending upon whether it maintains the following qualities:

- One complete module for each new type, no two new types in one module

(Score:

$$8 \sum_{modules} 1/$$

number of types in the module+

$$8 \sum_{classes} 1/$$

number of modules per class)

- A single header for each new type (Score:

$$4 \sum_{headers} 1/$$

number of types in the header)

- No imperative code in that header (Score: 2 or 0 per header)
- Minimal number of classes that need to have their implementation changed if the base list type is implemented using various storage structures (automatic array, dynamically allocated array, linked list, doubly linked list, etc.) (Score: $4 \times (\text{number of unchanged classes} / \text{total number of classes})$)

Also demonstrate the maximum score you could get, and what changes you could do in your design and/or implementation if you are not reaching that score.

Alongside, you report your effort throughout, along the following lines, **if you have not done so already in your submission**.

1. **The top level:** toolchains offered by your implementation, including the front end
2. **The base:** the foundation classes and/or templates from the standard libraries used, any design frameworks and boilerplate code used or downloaded from some copilot or similar tools, etc.
3. **The manufacturing:** the IDE chosen, the development and testing process (cycle, if cyclic).
4. **The Team:** If you choose to work in a team of more than one, then report the details, roles, onboarding, division of labour planned, actual contributions.

Maximum file size: 512 MB, maximum number of files: 1





Files



You can drag and drop files here to add them.

Accepted file types

PDF document .pdf

[◀ Last Year's Data Abstraction and Object Orientation Tutorials](#)

[Jump to activity](#)

[Tasks for the End of the OO Module ▶](#)